

Immersion Practice Session[2 Weeks]: ICP batch

Day	Topics
1	Java 8+ (Functional Programming)
2	Collections, Streams & Interview Modern Programming
3	Advanced Arrays, Two Pointers, Sliding Window
4	Binary Search Patterns, Prefix Sum, Hashing,
5	Stack, Queue, Monotonic Stack
6	Trees & BST
7	Graph Algorithms
8	Greedy + Heap
9	Dynamic Programming
10	Mock Contest + Interview Discussion

Day 1 – Functional Programming Fundamentals

- Functional Programming vs OOP
- Functional Interface
- Lambda Expressions
- Method References
- Built-in Functional Interfaces
 - Predicate
 - Function
 - Consumer
 - Supplier
 - UnaryOperator
 - BinaryOperator
- Optional API

Day 2 – Streams API

- Stream Creation
- filter()
- map()
- flatMap()
- distinct()
- sorted()
- limit(), skip()
- peek()
- reduce()
- collect()
- Collectors
 - toList()
 - groupingBy()
 - partitioningBy()
 - counting()
 - joining()
 - mapping()
- Comparator.comparing()
- Chaining Streams
- Real Interview Problems

Interview Questions

- Find duplicate elements using Streams.
- Group employees by department.

- Find the second highest salary.
- Frequency of each character.
- First non-repeating character.
- Partition even and odd numbers.
- Top K frequent elements.
- Convert List to Map.
- Remove duplicates while preserving order.
- Merge two lists without duplicates.

Learning

Topics by requirements

Why Java 8?

Before learning Lambda, you must understand **why Java changed after almost 20 years.**

Java before Java 8

Imagine you have a list of students.

```
class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}
```

Suppose you want to print only students with marks greater than 80.

Traditional Way

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        students.add(new Student("Amit", 70));
        students.add(new Student("Rahul", 90));
        students.add(new Student("Neha", 85));
        students.add(new Student("Riya", 60));

        for (Student s : students) {
            if (s.marks > 80) {
                System.out.println(s.name);
            }
        }
    }
}

class Student {

    String name;
    int marks;
    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}
```

```
}
```

Output

Rahul

Neha

Looks fine, right?

Now let's increase the requirements.

Requirement 2

Print students

- marks > 80
- sort by name
- convert names to uppercase
- print only first 2 students

Now imagine writing all of this with loops.

Loop

↓

if

↓

Another List

↓

Sorting

↓

Another Loop

↓

Uppercase

↓

Counter

↓

Print

Code becomes longer and harder to maintain.

This is exactly why Java 8 was introduced.

Java developers asked:

"Can we process collections in a cleaner, more expressive way?"

The answer was:

- Lambda Expressions
- Streams
- Functional Interfaces

Code:

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class StudentList {  
    public static void main(String[] args) {  
        List<Student> list = new ArrayList<>();  
  
        list.add(new Student(10, "Raju"));  
        list.add(new Student(23, "Krishna"));  
        list.add(new Student(13, "Samad"));  
        list.add(new Student(14, "Rakesh"));  
    }  
}
```

```

        // display name only marks > 15

        // for(Student s: list){
        //     if(s.marks > 15){
        //         System.out.println(s.name);
        //     }
        // }

        list.stream().filter(s->s.marks>15).forEach(s->System.out.println(s.name));

        // System.out.println(list);

    }
}

class Student{
    int marks;
    String name;

    Student(int marks, String name){
        this.marks = marks;
        this.name = name;
    }
}

```

Requirement-2

```

students.stream()
    .filter(s -> s.marks > 80)
    .sorted((s1, s2) -> s1.name.compareTo(s2.name))
    .map(s -> s.name.toUpperCase())
    .limit(2)
    .forEach(System.out::println);

```

Another Example

Suppose we want to sort numbers.

Traditional Java

```

public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(8, 3, 1, 9, 5);
        Collections.sort(list);
        System.out.println(list);
    }
}

```

Output

[1, 3, 5, 8, 9]

Easy.

Now suppose we want **descending order**.

Before Java 8, many developers wrote a separate Comparator.

```

import java.util.*;

```

```

class DescComparator implements Comparator<Integer> {
    @Override
    public int compare(Integer a, Integer b) {
        return b - a;
    }
}

public class Main {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(8, 3, 1, 9, 5);
        Collections.sort(list, new DescComparator());
        System.out.println(list);
    }
}

```

Notice the pain:

- Create a new class
- Implement Comparator
- Override compare()
- Write several lines for a simple comparison

Java 8 allows the same logic using a Lambda:

```
Collections.sort(list, (a, b) -> b - a);
```

One line instead of an entire class.

Another Real Example

Suppose you create a button in a GUI application.

Before Java 8:

```

button.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button Clicked");
    }

});

```

Java 8:

```
button.addActionListener(e -> System.out.println("Button Clicked"));
```

1. Lambda Expression

Definition

A **Lambda Expression** is a concise way to represent an **anonymous function** (a function without a name) that can be passed as an argument or stored in a variable.

It provides the implementation of a **Functional Interface** without creating a separate class or anonymous class.

Simple Definition (Interview)

A Lambda Expression is a short way to implement a Functional Interface in Java.

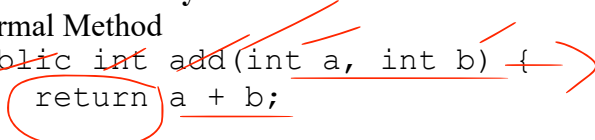
Lambda = Anonymous Method

Normal Method

```

public int add(int a, int b) {
    return a + b;
}

```



Lambda Expression

(a, b) -> a + b

Notice

Method Name	✗	Removed	
Return Type	✗	Removed	
Access Modifier	✗	Removed	
Curly Braces	✗	(Sometimes)	
return	✗	(Sometimes)	

Only Logic ✓

Lambda Syntax

(parameters) -> expression

OR

**(parameters) -> {
 statements
}**

How to Convert Normal Method into Lambda

Suppose method is

```
public int add(int a, int b) {  
    return a + b;  
}
```

Step 1

Remove Method Name

```
public int (int a, int b) {  
    return a + b;  
}
```

Step 2

Remove Access Modifier

```
int (int a, int b) {  
    return a + b;  
}
```

Step 3

Remove Return Type

Compiler already knows from Functional Interface.

```
(int a, int b) {  
    return a + b;  
}
```

Step 4

Replace

```
{  
with  
->  
(int a, int b) -> {  
    return a + b;  
}
```

Step 5

Remove Data Types (Optional)

```
(a, b) -> {  
    return a + b;  
}
```

Step 6

If only one return statement

Remove braces

Remove return

```
(a, b) -> a + b
```

🎉 Final Lambda

Conversion Example 1

Normal Method

```
public void print() {  
    System.out.println("Hello");  
}
```

↓

Remove name

```
public void () {  
    System.out.println("Hello");  
}
```

↓

Remove modifier

```
void () {  
    System.out.println("Hello");  
}
```

↓

Remove return type

```
() {  
    System.out.println("Hello");  
}
```

↓

Replace with Arrow

```
() -> {  
    System.out.println("Hello");  
}
```

↓

Single statement

```
() -> System.out.println("Hello");
```

Conversion Example 2

Normal Method

```
public int square(int x) {  
    return x * x;  
}
```

↓

Lambda

```
x -> x * x
```

OR

(x) -> x * x

Conversion Example 3

Normal Method

```
public int max(int a, int b) {  
    return a > b ? a : b;  
}
```

↓

Lambda

(a, b) -> a > b ? a : b

Conversion Example 4

Normal Method

```
public String fullName(String first, String last) {  
    return first + " " + last;  
}
```

↓

Lambda

(first, last) -> first + " " + last

Rule: Multiple statements hone par { } aur return remove nahi kar sakte.

Shortcut Rules

No Parameter

() -> System.out.println("Hello");

One Parameter

x -> x * x

OR

(x) -> x * x

Two Parameters

(a, b) -> a + b

Multiple Statements

```
(a, b) -> {  
  
    int c = a + b;  
  
    return c;  
}
```

Teaching Tip (Very Effective)

Board par ye 6-step formula likho aur har example isi se convert karvao:

Normal Method

↓

Remove Method Name

↓

Remove Access Modifier

↓

Remove Return Type (**type inference**)

↓

Replace { with ->

↓

Remove Parameter Types (**type inference**)

↓

If only one return statement:

Remove { }, remove return

↓

Lambda Expression

Practice Questions

Level 1

Write Lambda Expressions for:

1. Print "Hello Java"
2. Square of a number
3. Cube of a number
4. Sum of two integers
5. Maximum of two numbers

Level 2

Convert Anonymous Class into Lambda

Question 1

```
Runnable r = new Runnable() {  
  
    @Override  
    public void run() {  
  
        System.out.println("Running");  
  
    }  
  
};
```

Question 2

```
Comparator<Integer> c = new Comparator<Integer>() {  
  
    @Override  
    public int compare(Integer a,Integer b) {  
  
        return a-b;  
  
    }  
  
};
```

Level 3

Create Lambda for

1. Find larger number
2. Check Even/Odd
3. Check Prime
4. Find factorial

5. Reverse a string
6. Count vowels
7. Concatenate two strings
8. Find area of rectangle
9. Find simple interest
10. Find maximum among three numbers

Interview Questions

Q1. What is Lambda Expression?

Answer:

A Lambda Expression is a concise way to implement a Functional Interface using an anonymous function.

Q2. Which Java version introduced Lambda?

Answer: Java 8

Q3. Can Lambda be used with every interface?

Answer: No. Only with **Functional Interfaces** (interfaces having exactly one abstract method).

Q4. What are the main benefits of Lambda?

- Less code
- Improved readability
- Supports Functional Programming
- Enables Stream API
- Simplifies multithreading

Q5. Can Lambda access local variables?

Answer: Yes, but only **final** or **effectively final** local variables.

Q6. What is the difference between Anonymous Class and Lambda?

Anonymous Class	Lambda
Creates an anonymous class	Represents only the implementation of a functional method
Has its own <code>this</code> reference	Uses the enclosing class's <code>this</code>
More boilerplate	Less boilerplate
Can implement any interface or extend a class	Only works with Functional Interfaces

Coding Assignments

1. Create a `Calculator` functional interface and implement **Addition, Subtraction, Multiplication, and Division** using Lambda Expressions.
2. Sort a list of integers in ascending and descending order using Lambda.
3. Sort a list of students by **name**, **age**, and **marks** using Lambda and `Comparator`.
4. Print all even numbers from a list using `forEach()` and Lambda.
5. Create your own functional interface `MessagePrinter` and print custom messages using Lambda.

("Lambda Expressions are useless without Functional Interfaces, and Streams are impossible to understand without Predicate, Function, Consumer, etc.")

2. Functional Interface

A **Functional Interface** in Java is an interface that contains **exactly one abstract method**.

Because it has only one abstract method, it becomes the **target type** for **lambda expressions** and **method references** (as we discussed earlier). It is often referred to as a **SAM (Single Abstract Method)** interface.

The Golden Rule

One abstract method. Period.

(But it can have any number of default, static, or private methods.)

Built-in Functional Interfaces in Java (`java.util.function`)

Imagine a company.

Each employee has exactly **one responsibility**.

Employee	Responsibility
Predicate	Check something
Function	Convert something
Consumer	Use something
Supplier	Create something
Comparator	Compare two things

Each employee has only **one job**.

That is exactly why they have only **one abstract method**.

Interface	Input	Output	Purpose	Stream Method
Predicate	T	boolean	Test	filter()
Consumer	T	void	Consume	forEach()
Function<T,R>	T	R	Transform	map()
Supplier	None	T	Produce	Stream.generate()
Comparator	T,T	int	Compare	sorted()
UnaryOperator	T	T	Modify same type	map()
BinaryOperator	T,T	T	Combine	reduce()

The Big Picture

Everything in the Stream API is built on these interfaces:

Collection

|



stream()

|



filter() -----> Predicate<T>

|



map() -----> Function<T,R>

|



sorted() -----> Comparator<T>

|



forEach() -----> Consumer<T>

|



collect()

Notice something beautiful:

- `filter()` asks a **question** → Predicate
- `map()` **transforms** data → Function
- `sorted()` **compares** objects → Comparator
- `forEach()` **uses** each object → Consumer

3. Anonymous Class

An **anonymous class** in Java is an **inner class without a name** that is declared and instantiated in a **single expression** using the `new` keyword.

You use it when you need a one-off implementation of an **interface** or an **abstract/concrete class** right at the point of use, without formally creating a separate named `.java` file.

Basic Syntax

```
new InterfaceOrClassName() {  
    // Method implementations and fields go here  
};
```

(Note the required semicolon at the end, because it's a single statement.)

Suppose we have an interface.

```
interface Greeting {  
  
    void sayHello();  
  
}
```

Can we create its object?

```
Greeting g = new Greeting();
```

✗ No.

Output

Cannot instantiate the type Greeting

Because **interfaces don't have implementations**.

Solution 1: Create a Separate Class

```
interface Greeting {  
  
    void sayHello();  
  
}  
  
class EnglishGreeting implements Greeting {  
  
    @Override  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
  
}  
  
Use  
Greeting g = new EnglishGreeting();
```

```
g.sayHello();
```

Output

Hello

Problem

Suppose we need this implementation only **once**.

Creating a new file

EnglishGreeting.java

just for one method feels unnecessary.

Java developers thought:

Can we create the implementation on the spot?

The answer was **Anonymous Inner Class**.

Anonymous Class

```
Greeting g = new Greeting() {  
  
    @Override  
    public void sayHello() {  
  
        System.out.println("Hello");  
  
    }  
  
};
```

```
g.sayHello();
```

Output

Hello

Notice

We never wrote

```
class EnglishGreeting
```

Java created an unnamed class automatically.

That's why it's called an **Anonymous Class**.

What's Happening Behind the Scenes?

You write

```
Greeting g = new Greeting() {  
  
    @Override  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
  
};
```

The compiler internally generates something similar to:

```
class Main$1 implements Greeting {  
  
    @Override  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
  
}
```

Then

```
Greeting g = new Main$1();
```

So an anonymous class is simply a **compiler-generated class without a programmer-defined name**.

Memory Diagram

Greeting g



Anonymous Class Object

sayHello()

Another Example

Runnable

Normally

```
class MyTask implements Runnable {  
  
    @Override  
    public void run() {  
        System.out.println("Running...");  
    }  
  
}
```

Use

```
Runnable r = new MyTask();
```

```
r.run();
```

Anonymous Class

```
Runnable r = new Runnable() {  
  
    @Override  
    public void run() {  
  
        System.out.println("Running...");  
  
    }  
  
};
```

```
r.run();
```

Output

Running...

Comparator Example (Very Important)

Before Java 8

```
List<String> names = Arrays.asList(  
    "Rahul",  
    "Amit",  
    "Neha"  
);  
  
Collections.sort(names, new Comparator<String>() {  
  
    @Override  
    public int compare(String a, String b) {
```

```
        return a.compareTo(b);
    }
});
```

```
System.out.println(names);
```

Output

```
[Amit, Neha, Rahul]
```

Why Java 8 Introduced Lambda?

Look at this code:

```
new Comparator<String>() {

    @Override
    public int compare(String a, String b) {

        return a.compareTo(b);

    }

}
```

How much of it is actual business logic?

Only this line:

```
return a.compareTo(b);
```

Everything else is boilerplate.

Java 8 Lambda

```
Collections.sort(names,
    (a, b) -> a.compareTo(b));
```

Same output.

Much shorter.

Transformation

Step 1

Anonymous Class

```
new Comparator<String>() {

    @Override
    public int compare(String a, String b) {

        return a.compareTo(b);

    }

}
```

↓

Step 2

Remove

```
new Comparator<String>()
```

↓

Step 3

Remove

@Override

↓

Step 4

Remove

compare()

↓

Step 5

Replace with

(a, b) ->

Final

(a, b) -> a.compareTo(b)

Why Only Functional Interfaces?

Suppose

```
interface Demo {
```

```
    void show();
```

```
    void print();
```

```
}
```

Can Java convert this into a lambda?

No.

Which method should the lambda implement?

show()

or

print()

Java can't decide.

So Lambda works **only** when there is exactly **one abstract method**.

That is called a **Functional Interface**.

Anonymous Class vs Lambda

Anonymous Class	Lambda Expression
Creates a new anonymous class	Doesn't create a separate anonymous class; represents an implementation of a functional interface
Verbose	Concise
Available before Java 8	Introduced in Java 8
Can implement interfaces with multiple abstract methods (by implementing all of them) or extend classes	Only works with functional interfaces (one abstract method)
Has its own <code>this</code> reference	<code>this</code> refers to the enclosing class
More boilerplate	Minimal syntax

Interview Question

Q. Is Lambda just a shorter Anonymous Class?

Answer:

Conceptually, yes—both provide an implementation of an interface without writing a separate named class. Technically, no—they are implemented differently by the JVM. A lambda is **not** compiled into an anonymous inner class in the traditional sense. Java uses the `invokedynamic` mechanism and runtime-generated function objects, which is generally more efficient than generating a separate anonymous class. So the mental model is:

Separate Class



Anonymous Class



Lambda Expression

As Java evolved, each step reduced boilerplate while expressing the same idea: **passing behavior instead of just data.**

4. Method Reference Operator in Java 8.

This code

```
students.stream()  
    .forEach(System.out::println);
```

Many students ask:

Sir, ye :: kya hai?

Answer:

:: means **Method Reference**.

It is simply a **shorter form of a Lambda Expression**.

Step 1: Traditional Way

Suppose we have

```
List<String> names = List.of("Rahul", "Neha", "Amit");
```

To print every name:

```
for(String name : names){  
    System.out.println(name);  
}
```

Step 2: Using Stream

```
names.stream()  
    .forEach(name -> System.out.println(name));
```

Here,

```
name -> System.out.println(name)
```

is a **Lambda Expression**.

Meaning:

Take one name

↓

Call `println(name)`

Step 3: Method Reference

Look carefully:

```
name -> System.out.println(name)
```

The lambda is **doing only one thing**:

Calling an existing method.

Since we're not adding any extra logic, Java lets us shorten it to:

```
System.out::println
```

So this:

```
.forEach(name -> System.out.println(name))
```

and this:

```
.forEach(System.out::println)
```

are exactly the same.

Visual Representation

Lambda

name



System.out.println(name)



Method Reference

System.out::println

Another Example

Suppose

```
class Student{

    String name;

    String getName(){
        return name;
    }

}
```

Lambda

student -> student.getName()

Method Reference

Student::getName

Exactly same.

Another Example

Lambda

s -> s.toUpperCase()

Method Reference

String::toUpperCase

Another Example

Lambda

x -> Math.abs(x)

Method Reference

Math::abs

Why does it work with **forEach()**?

Remember:

forEach()

expects a **Consumer<T>**.

Consumer interface:

```
@FunctionalInterface
interface Consumer<T>{

    void accept(T t);

}
```

So **forEach()** internally does something like this:

```
Consumer<String> consumer = System.out::println;
```

Then for every element:

```
consumer.accept(name);  
Which becomes  
System.out.println(name);
```

What exactly is **System.out**?

System is a class.

System

Inside it,

out

is a static object of type `PrintStream`.

`System.out`

is basically a `PrintStream` object.

`PrintStream` has a method

`println()`

So

`System.out::println`

means

"Whenever you get a value, call the `println()` method on the `System.out` object."

Memory View

names

|



"Rahul"

|



`forEach()`

|



`Consumer`

|



`System.out::println`

|



`System.out.println("Rahul")`

Then

"Neha"

↓

`System.out.println("Neha")`

Then

"Amit"

↓

`System.out.println("Amit")`

Rule to Remember

Use a **Method Reference** only when your lambda simply calls an existing method.

Lambda Expression	Method Reference
<code>x -> System.out.println(x)</code>	<code>System.out::println</code>
<code>s -> s.toUpperCase()</code>	<code>String::toUpperCase</code>

Lambda Expression	Method Reference
<code>student -> student.getName()</code>	<code>Student::getName</code>
<code>x -> Math.abs(x)</code>	<code>Math::abs</code>

If your lambda contains additional logic, you **cannot** replace it with a method reference.

For example:

```
x -> {
    System.out.println(x);
    System.out.println("Done");
}
```

This **cannot** become a method reference because there are two statements, not just a single existing method call.

This is why you'll see `System.out::println` very frequently with `forEach()`—it's the cleanest way to say: **"For every element, print it."**

Ye topic bhi agar sirf methods (`isPresent()`, `get()`, `orElse()`) se start karoge to students confuse ho jayenge. Best teaching sequence hai:

Problem → NullPointerException → Traditional Solution → Optional → Optional Methods → Interview Questions → Practice

5. Optional Class in Java

What is Optional?

Definition

Optional is a container object introduced in **Java 8** that may or may not contain a value.

Simple Definition

Optional is a wrapper object used to avoid NullPointerException.

Why was Optional Introduced?

Imagine a method

```
public String getName() {
    return null;
}
```

Calling

```
String name = getName();
System.out.println(name.length());
```

Output

```
Exception in thread "main"
```

```
java.lang.NullPointerException
```

Ask Students

Who made the mistake?

Nobody.

Method returned

```
null
```

Program crashed.

What is NullPointerException?

Definition

A **NullPointerException (NPE)** occurs when we try to access methods or properties of a null object.

Example

```
String name = null;
```

```
System.out.println(name.length());
```

Output

```
java.lang.NullPointerException
```

Traditional Solution

Before Java 8

```
String name = getName();  
if(name != null){  
    System.out.println(name.length());  
}
```

Problem

Imagine 100 variables.

```
if(name!=null)  
if(city!=null)  
if(address!=null)  
if(employee!=null)  
if(manager!=null)
```

Too many null checks.

Java 8 Solution

Instead of returning

String

Return

Optional<String>

Now method becomes

```
public Optional<String> getName(){  
  
    return Optional.of("Amir");  
  
}
```

or

```
return Optional.empty();
```

Now caller knows

Value may or may not exist.

Visual Representation

Traditional

Method

↓

Returns String

↓

Could be null

↓

NPE

Using Optional

Method

↓

Returns Optional<String>

↓
Value Exists?
↓
YES → Use it
NO → Handle gracefully

Creating Optional Objects

1. Optional.of()

Used when value is definitely NOT null.

```
Optional<String> name = Optional.of("Amir");
```

Output

```
Optional[Amir]
```

If null passed

```
Optional.of(null);
```

Output

```
NullPointerException
```

Rule

Never pass null to Optional.of()

2. Optional.ofNullable()

Used when value may or may not be null.

```
String name = null;
```

```
Optional<String> op = Optional.ofNullable(name);
```

Output

```
Optional.empty
```

If value exists

```
String name = "Amir";
```

```
Optional<String> op = Optional.ofNullable(name);
```

Output

```
Optional[Amir]
```

3. Optional.empty()

Creates an empty Optional.

```
Optional<String> op = Optional.empty();
```

Output

```
Optional.empty
```

Checking Value

isPresent()

```
Optional<String> op = Optional.of("Java");
```

```
System.out.println(op.isPresent());
```

Output

```
true
```

Empty

```
Optional.empty().isPresent();
```

Output

```
false
```

get()

Returns value.

```
Optional<String> op = Optional.of("Java");
```

```
System.out.println(op.get());
```

Output

Java

But

```
Optional.empty().get();
```

Output

NoSuchElementException

Interview Point

Never call get() without checking.

ifPresent()

Instead of

```
if (op.isPresent()) {
```

```
    System.out.println(op.get());
```

```
}
```

Use

```
op.ifPresent(System.out::println);
```

Cleaner.

orElse()

Return default value.

```
Optional<String> op = Optional.empty();
```

```
System.out.println(op.orElse("Unknown"));
```

Output

Unknown

If value exists

```
Optional.of("Java").orElse("Unknown");
```

Output

Java

orElseGet()

Default value generated using Supplier.

```
String value = op.orElseGet(() -> "Default Name");
```

Difference

orElse()

↓

Default object already created

orElseGet()

↓

Creates only if required

orElseThrow()

Throw custom exception.

```
String name = op.orElseThrow(  
    () -> new RuntimeException("Name Missing")  
);
```

map()

Transform value.

```
Optional<String> op = Optional.of("java");  
Optional<String> upper =  
    op.map(String::toUpperCase);
```

```
System.out.println(upper.get());
```

Output

JAVA

filter()

```
Optional<String> op =  
    Optional.of("Java");  
Optional<String> ans =  
    op.filter(s -> s.startsWith("J"));
```

Output

Optional[Java]

If false

Optional.empty

Complete Flow

Create Optional

↓

Check Value

↓

Transform

↓

Return Default

↓

Throw Exception

Real Example

```
import java.util.Optional;
```

```
public class Main {
```

```
    static Optional<String> findStudent(int id) {  
        if(id==101)  
            return Optional.of("Amir");  
        return Optional.empty();  
    }
```

```
    public static void main(String[] args) {
```

```
        String name =  
            findStudent(100)  
                .orElse("Student Not Found");
```

```
        System.out.println(name);
```



```
}  
}
```

Output

Student Not Found

Interview Questions

What is Optional?

A wrapper class introduced in Java 8 to avoid NullPointerException.

Difference

<code>of()</code>	<code>ofNullable()</code>
Null not allowed	Null allowed

Difference

<code>orElse()</code>	<code>orElseGet()</code>
Always creates default value	Creates only when needed

Why should `get()` be avoided?

Because it throws

`NoSuchElementException`

when `Optional` is empty.

Is `Optional` Serializable?

No.

Can `Optional` contain null?

No.

Empty means

`Optional.empty()`

not

`Optional(null)`

Practice Questions

Easy

1. Create `Optional` using `of()`.
 2. Create `Optional` using `ofNullable()`.
 3. Check value using `isPresent()`.
 4. Print using `ifPresent()`.
 5. Return default using `orElse()`.
-

Medium

6. Convert string to uppercase using `map()`.
 7. Filter strings starting with "A".
 8. Use `orElseThrow()` to throw an exception.
 9. Find employee by ID using `Optional`.
 10. Chain `map()` and `orElse()` together.
-

Summary Table

Method	Purpose
<code>of()</code>	Create <code>Optional</code> (non-null value)
<code>ofNullable()</code>	Create <code>Optional</code> (null allowed)
<code>empty()</code>	Create empty <code>Optional</code>

Method	Purpose
<code>isPresent()</code>	Check if value exists
<code>get()</code>	Retrieve value (use carefully)
<code>ifPresent()</code>	Execute only if value exists
<code>orElse()</code>	Return default value
<code>orElseGet()</code>	Lazily generate default value
<code>orElseThrow()</code>	Throw custom exception if empty
<code>map()</code>	Transform the contained value
<code>filter()</code>	Keep value only if condition matches

Students usually remember Optional much better if you teach it as a **replacement for repetitive null checks**. Build the lecture around this progression:

Null Value

↓

NullPointerException

↓

`if (obj != null)`

↓

Too many null checks

↓

Optional (Java 8)

↓

Handle missing values safely and cleanly

This narrative makes every Optional method (`of`, `ofNullable`, `orElse`, `map`, etc.) feel like a natural solution rather than just another API to memorize.